3D File Format Comparison: Detailed Explanation

Overview

This document expands upon the summary comparison table of common 3D file formats by providing detailed descriptions of each attribute: Animation, Textures, Compression, Human-readability, and Ideal Use Cases. Understanding these characteristics helps developers and artists choose the right format for their pipeline or platform.

Key Attributes Explained

- **Animation:** Indicates whether the format supports storing animation data (e.g., skeletal rigs, keyframes, morph targets).
- **Textures:** Refers to the ability to link or embed texture maps such as diffuse, normal, roughness, or PBR materials.
- **Compression:** Describes whether the format natively supports data compression for smaller file sizes.
- **Human-readable:** Describes whether the format can be easily opened and read in a text editor (useful for debugging or manual edits).
- **Ideal For:** Suggests common workflows or industries where the format is a strong fit.

Format-by-Format Breakdown

OBJ - **Animation:** No animation support; only static geometry. - **Textures:** Supports basic texture references via `.mtl` files. - **Compression:** None. - **Human-readable:** Yes, fully text-based. - **Ideal For:** Static mesh exchange, simple geometry tasks, educational use.

FBX - **Animation:** Fully supports animations, including bones, skinning, and blendshapes. - **Textures:** Rich material and texture support. - **Compression:** Supports binary encoding with compression. - **Human-readable:** ASCII version is partially readable; binary version is not. - **Ideal For:** Game development, animated characters, interoperability between DCC tools.

GLTF / GLB - **Animation:** Yes; supports skeletal animations, morph targets. - **Textures:** Yes; excellent PBR support and efficient embedding. - **Compression:** GLB is binary and compressed; GLTF can use external compression methods like Draco. - **Human-readable:** GLTF is JSON (readable); GLB is binary. - **Ideal For:** Web delivery, AR/VR, real-time applications.

STL - **Animation:** None. - **Textures:** None. - **Compression:** Binary STL files are compact. - **Human-readable:** ASCII STL is readable; binary STL is not. - **Ideal For:** 3D printing, CAD export.

DAE (COLLADA) - **Animation:** Supports skeletal animations, keyframes. - **Textures:** Supports textures and complex material data. - **Compression:** None natively. - **Human-readable:** XML-based, fully readable. - **Ideal For:** Interchange between modeling tools; intermediate format.

USD / USDZ - **Animation:** Yes; very robust support for rigging, motion, and layered animation. - **Textures:** Full material system and references. - **Compression:** Yes; efficient for complex scenes. - **Human-readable:** No (binary format). - **Ideal For:** VFX pipelines, collaborative workflows, Apple ARKit.

Comparison Table

Format	Animation	Textures	Compression	Human-readable	Ideal For
OBJ	No	Yes	No	Yes	Basic geometry
FBX	Yes	Yes	Yes	Partially	Games, animation
GLTF	Yes	Yes	Yes	Partially	Web, real-time
STL	No	No	Yes	Yes/No	3D printing
DAE	Yes	Yes	No	Yes (XML)	DCC interoperability
USD	Yes	Yes	Yes	No	Complex scene pipelines

Conclusion

Each 3D format serves different purposes. While `.obj` is excellent for simplicity and universality, formats like `.gltf` and `.usd` are better suited for modern, feature-rich applications. The choice depends on whether your priority is interoperability, real-time performance, animation support, or scene complexity.